

# DebtPulse — Interview Preparation Guide

A single reference for explaining the DebtPulse project and its concepts in an interview. Every topic has a **standard explanation** plus "In DebtPulse" — where and how it is actually used, with file references.

## 0. Tech stack (what & where)

| Layer               | Technology                                                            | Where used in DebtPulse                                                                                         |
|---------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Frontend            | React 18 + Vite 5                                                     | SPA in <code>frontend/</code> (JS) and <code>frontend-ts/</code> (TypeScript)                                   |
| Routing             | react-router-dom 6                                                    | <code>App.jsx</code> routes, <code>ProtectedRoute.jsx</code> guard                                              |
| State               | Redux Toolkit 2 + react-redux 9; Context API                          | <code>store/notificationsSlice.js</code> (badge);<br><code>auth/AuthContext.jsx</code> (session)                |
| HTTP                | Axios 1.7                                                             | <code>api/client.js</code> (interceptors), <code>api/services.js</code> (endpoints)                             |
| UI                  | react-bootstrap 2 + Bootstrap 5.3 + Recharts                          | shared components, <code>styles/theme.css</code> , analytics charts                                             |
| Testing             | Vitest + React Testing Library                                        | <code>*.test.jsx</code> , <code>src/test/setup.js</code>                                                        |
| Backend             | Java 21 + Spring Boot 3.3                                             | all <code>*-service</code> modules                                                                              |
| Microservices       | Spring Cloud: Gateway, Eureka, Config Server, OpenFeign, LoadBalancer | <code>api-gateway</code> , <code>eureka-server</code> , <code>config-server</code> , <code>config-repo/</code>  |
| Resilience          | Resilience4j                                                          | Feign fallbacks (e.g., <code>AccountClientFallback</code> )                                                     |
| Security            | Spring Security + JWT (jjwt)                                          | gateway <code>JwtAuthenticationGlobalFilter</code> , <code>common-lib</code> <code>RoleBasedHeaderFilter</code> |
| Persistence         | Spring Data JPA / Hibernate, MySQL, Flyway                            | each service's <code>repository/</code> , <code>entity/</code> , migrations                                     |
| Mapping/boilerplate | MapStruct, Lombok                                                     | mappers, entities/DTOs                                                                                          |
| Docs                | springdoc-openapi (Swagger UI)                                        | per-service <code>/swagger-ui</code>                                                                            |
| Shared              | common-lib                                                            | security filter, <code>AuthContext</code> , audit aspect, enums, DTOs                                           |

## 1. Project flow

### a. Microservices flow

The browser only talks to the **API Gateway** ( `http://localhost:9090` ). The gateway validates the JWT and routes each `/api/` path to the right service via Eureka **service discovery** ( `lb://service-name` ). Services get their config from the Config Server ( `config-repo/` ), call each other with OpenFeign (guarded by Resilience4j fallbacks), and each owns its MySQL\*\* schema.

```
React SPA → API Gateway (JWT + routing) → [auth | account | contact | field | settlement | legal | analytics | notification]
                                     | Feign (lb://) between services
                                     └─ each service → its own MySQL schema
Eureka (registry) · Config Server (config-repo) · common-lib (shared code)
```

### b. Frontend flow

`main.jsx` mounts the app inside providers: **Redux Provider** → **BrowserRouter** → Preferences/Toast → **AuthProvider**. `App.jsx` declares routes; protected routes are wrapped in `ProtectedRoute`. A page calls a function in `api/services.js`, which uses the shared **axios** instance ( `api/client.js` ); the response updates component state and re-renders.

### c. Complete request flow (example)

ContactWorkspace → contactApi.create() → axios attaches Bearer <JWT> → gateway validates + injects X-Auth-\* → contact-service ( @PreAuthorize + @Valid ) → business rules + Feign call to account-service → save to MySQL → map entity to DTO → 201 back → UI toasts and reloads.

## 2. React concepts

**Why React (features).** Component-based, declarative UI; a **Virtual DOM** for efficient updates; one-way data flow; huge ecosystem; hooks for logic reuse. In DebtPulse it lets us compose one shared DataTable , FormModal , Field across every module.

**Functional components.** Plain functions returning JSX; state/lifecycle via hooks (no classes). \*Every\* component in DebtPulse is functional (e.g., AccountsPage , Layout , ProtectedRoute ).

**Props.** Read-only inputs passed parent→child. E.g., <DataTable columns={...} page={...} onClick={...} /> , <Field label name value onChange error /> .

**Virtual DOM.** React keeps an in-memory tree, diffs the new tree against the old on state change ("reconciliation"), and patches only what changed in the real DOM — fast and predictable.

**React Router.** Declarative client-side routing — the URL maps to components without full page reloads (see topic 16).

**Redux.** Predictable centralized store for global state (topic 13). DebtPulse uses it in one place — the notification badge — as a clean showcase.

**react-dom.** The renderer that mounts React onto the browser DOM:

```
ReactDOM.createRoot(document.getElementById('root')).render(<App/>) in main.jsx .
```

**Axios & how the frontend knows the gateway port.** The axios baseUrl is the **relative** path /api ( api/client.js ). In dev, Vite proxies /api to the gateway at http://localhost:9090 ( vite.config.js → server.proxy ), so the browser stays same-origin (no CORS). In production the SPA is served behind the same gateway/ingress, so /api is already same-origin. The port lives in **one** place (the Vite proxy / VITE\_API\_TARGET ), not scattered in code.

```
// vite.config.js
server: { port: 5173, proxy: { '/api': { target: process.env.VITE_API_TARGET || 'http://localhost:9090', changeOrigin: true } } }
```

## 3. Node.js vs npm

- **Node.js** — the JavaScript **runtime** (V8 engine) that executes JS outside the browser; it runs the Vite dev server, the build, and the tests.
- **npm** — the **package manager** bundled with Node: installs dependencies ( npm install ), runs scripts ( npm run build , npm test ) from package.json .

One-liner: \*Node runs JS; npm manages the packages and scripts that JS project needs.\*

## 4. Dependencies installed

**Frontend ( package.json ).** Runtime: react , react-dom , react-router-dom , @reduxjs/toolkit , react-redux , axios , react-bootstrap , bootstrap , bootstrap-icons , recharts . Dev: vite , @vitejs/plugin-react , vitest , @testing-library/react , @testing-library/jest-dom , jsdom .

**Backend (Maven, per service).** spring-boot-starter-web , -data-jpa , -security , -validation , -actuator , -aop ; Spring Cloud: starter-netflix-eureka-client , starter-config , starter-openfeign , starter-loadbalancer , starter-circuitbreaker-resilience4j ; mysql-connector-j , flyway-core / flyway-mysql ; springdoc-openapi-starter-webmvc-ui ; lombok , mapstruct ; jjwt (JWT); test: spring-boot-starter-test (JUnit 5 + Mockito).

## 5. Axios vs fetch (and why axios)

|                           | fetch (built-in)                               | axios (used)                            |
|---------------------------|------------------------------------------------|-----------------------------------------|
| JSON                      | manual <code>res.json()</code>                 | auto-parsed <code>res.data</code>       |
| Errors                    | only rejects on network error (not on 4xx/5xx) | rejects on 4xx/5xx too                  |
| Interceptors              | none                                           | request/response interceptors           |
| Base URL / timeout        | manual                                         | built-in                                |
| Cancellation, credentials | verbose                                        | simple ( <code>withCredentials</code> ) |

**Why axios in DebtPulse:** interceptors are essential — one place attaches the `Bearer` token to every request, and one response interceptor does the silent **401** → **refresh** → **retry** and normalises the backend error envelope. `baseUrl`, `timeout`, and `withCredentials` (for the httpOnly refresh cookie) are set once. That would be a lot of repetitive code with raw `fetch`.

```
// api/client.js
api.interceptors.request.use((c) => { if (token) c.headers.Authorization = `Bearer ${token}`; return c; });
api.interceptors.response.use(r => r, async (err) => { /* 401 → doRefresh() → retry once */ });
```

## 6. Security in microservices

- **Single choke point:** the gateway is the only JWT validator ( `JwtAuthenticationGlobalFilter` ). It verifies signature + expiry once.
- **Trusted identity propagation:** the gateway **strips** any inbound `X-Auth-*` headers (anti-spoofing) and re-injects the authenticated `X-Auth-UserId/Role/BranchId/Name`. `common-lib`'s `RoleBasedHeaderFilter` rebuilds the Spring Security context from them, so services enforce `@PreAuthorize("hasRole('...')")` without re-checking the JWT.
- **Token model:** short-lived **access token** (JWT) kept in memory on the client; long-lived **refresh token** as an `httpOnly, Secure, SameSite=Strict` cookie (JS can't read it → XSS-resistant). Access token auto-refreshes silently.
- **Least privilege / ownership:** method-level RBAC per endpoint; officers can only act on accounts allocated to them (ownership guards).
- **Internal-only APIs:** service-to-service endpoints live under `/api/internal` and are not exposed through the public gateway routes.
- **Password safety:** BCrypt hashing, login lockout after repeated failures ( `AuthServiceImpl` ).
- **Auditing:** state-changing actions are recorded centrally (audit aspect in `common-lib`).

## 7. Bootstrap

**Bootstrap 5.3** is a CSS/component framework (grid, utilities, components). **react-bootstrap** wraps those components as React components ( `<Button>`, `<Modal>`, `<Table>`, `<Dropdown>` ). DebtPulse imports Bootstrap CSS in `main.jsx`, uses react-bootstrap throughout, and layers a bank-style theme + responsive tweaks in `styles/theme.css` (including the mobile "stacked table" cards via `.dp-stack` ).

## 8. TypeScript vs JavaScript

- **JavaScript** — dynamically typed; errors surface at runtime.
- **TypeScript** — JS + **static types**; catches type errors at **compile time**, gives editor autocomplete/refactoring, then compiles to JS.

In DebtPulse there are two parallel editions kept in parity: **frontend/ (JavaScript)** and **frontend-ts/ (TypeScript)**. The TS edition adds interfaces/types (e.g., `Account`, `PageResponse<T>`, `Role`) and generic components ( `DataTable<T>` ), so mismatched data shapes fail at build time.

## 9. Docker basics

- **Image** — an immutable, layered blueprint (app + deps + runtime). **Container** — a running instance of an image. **Dockerfile** — the recipe to build an image. **docker-compose** — declares and runs multiple containers together.
- **Why for microservices:** each service ships as its own image → identical run everywhere, independent scaling, isolation.

- **How DebtPulse maps to it:** each Spring Boot service is a self-contained runnable JAR → one image per service (a small JDK base + the jar); the React app builds to static files served by nginx; MySQL, Eureka, and Config Server run as their own containers. `docker-compose up` would start the whole system with a shared network, so `lb://` discovery and gateway routing work between containers. (Locally we run them directly via the IDE/Maven; the containerisation is the same topology.)

## 10. Spring Boot — annotations & JPA

### Common annotations (with where):

- `@SpringBootApplication` — each service's main class.
- `@RestController`, `@RequestMapping`, `@GetMapping`/`@PostMapping`/`@PutMapping`/`@PatchMapping`/`@DeleteMapping` — controllers (e.g., `SettlementController`).
- `@RequestBody`, `@PathVariable`, `@RequestParam`, `@Valid` — binding + validation of requests.
- `@PreAuthorize` — method-level RBAC.
- `@Service`, `@Component`, `@Repository`, `@Configuration` — Spring beans/layers.
- constructor injection (Spring injects dependencies) — every `*ServiceImpl`.
- `@Transactional` — atomic units (e.g., `onboard`, `settlement create`).
- `@Entity`, `@Id`, `@Column`, `@Enumerated`, `@ManyToOne` — JPA mappings (e.g., `SettlementProposal`, `CourtHearing`).
- `@FeignClient` — declarative HTTP client ( `AccountClient` ).
- `@Scheduled` — cron jobs ( `EscalationScheduler` ).

### JPA vs Spring Data JPA.

- **JPA** is the \*specification\* (interfaces/annotations for ORM); **Hibernate** is the implementation that maps objects↔tables and generates SQL.
- **Spring Data JPA** is a \*layer on top\* that removes boilerplate: you declare a repository **interface** and Spring generates the implementation. Derived queries ( `findByStatusAndAssignedAgentIdIsNull` ), paging, and `JpaSpecificationExecutor` for dynamic filters — no hand-written DAO.

```
public interface SettlementProposalRepository extends PagingAndSortingRepository<SettlementProposal,String>,
    CrudRepository<SettlementProposal,String> {
    boolean existsByAccountIdAndStatusIn(String accountId, Collection<SettlementStatus> statuses); // auto-implemented
}
```

## 11. ES6 features (incl. arrow functions)

Used throughout DebtPulse:

- **Arrow functions** — concise, lexical `this`: `const inr = (n) => '₹' + n.toLocaleString('en-IN')`.
- `let / const`, template literals: ``Bearer ${token}``.
- **Destructuring**: `const { data } = await authApi.login(...), ({ role } = useAuth())`.
- **Spread/rest**: `{ ...v, ptpAmount: Number(v.ptpAmount) }`.
- **Modules** `import / export`, **Promises / async-await** (all API calls), **default + optional chaining** `data?.content`, `??`.
- Array helpers `map/filter/reduce/some` (rendering rows, filtering lists).

## 12. AuthContext

`auth/AuthContext.jsx` is a **React Context** that holds the session and auth actions, so any component can read them via `useAuth()` without prop-drilling.

- State: `user`, `role`, `isAuthenticated`, `ready`.
- Actions: `login(email,password)` (calls `authApi.login`, stores token in memory, sets user), `logout()`.
- **Bootstrap**: on load it calls a silent `doRefresh()` (cookie-based) to restore the session after a page reload; `ready` gates routing until that resolves.

```
const { user, role, isAuthenticated, ready, login, logout } = useAuth();
```

## 13. useContext, useState — and why Redux differs

- `useState` — \*local\* component state; re-renders that component on change. Used everywhere for form/modal/filter state.
- `useContext` — reads a Context value (e.g., `useAuth()` reads `AuthContext`); good for app-wide values shared by many components (auth, theme, toasts).
- **Redux (Toolkit)** — a \*single external store\* with actions/reducers/selectors and dev-tools/time-travel; state lives outside the component tree and any component can read/update it predictably.

**When which:** `useState` for local UI; **Context** for simple shared values that change rarely (our **auth/session, preferences, toasts**); **Redux** for state many unrelated components read and write frequently. DebtPulse deliberately keeps Redux to **one** slice — the **notification unread badge** ( `store/notificationsSlice.js` ): the `Layout` polling effect `dispatch(setUnread(n))` and the topbar bell reads it with `useSelector`. Everything else is Context/local, which is the pragmatic choice for this app's size.

```
// store/notificationsSlice.js (Redux Toolkit)
createSlice({ name: 'notifications', initialState: {unread: 0},
  reducers: { setUnread: (s, a) => {s.unread = a.payload;}, clearUnread: (s) => {s.unread = 0; } } });
```

## 14. Auth guard

`auth/ProtectedRoute.jsx` is the route guard. It (1) waits for the session bootstrap ( `ready` ) so a reload doesn't wrongly bounce a logged-in user, (2) redirects to `/login` if not authenticated, and (3) redirects to `/forbidden` if the user's role isn't in the route's `allow` list. Used to wrap the whole authenticated `Layout` and each role-restricted route in `App.jsx`.

```
if (!ready) return <Spinner/>;
if (!isAuthenticated) return <Navigate to="/login" replace state={{ from: location }} />;
if (allow && !hasAny(role, allow)) return <Navigate to="/forbidden" replace />;
return children;
```

## 15. Each hook and its purpose (as used in DebtPulse)

| Hook                         | Purpose                               | Where in DebtPulse                                                         |
|------------------------------|---------------------------------------|----------------------------------------------------------------------------|
| <code>useState</code>        | local reactive state                  | forms, modals, filters (every page)                                        |
| <code>useEffect</code>       | run side effects on mount/deps change | data fetch, 30s notification poll ( <code>Layout</code> )                  |
| <code>useCallback</code>     | memoise a function identity           | stable <code>fetcher</code> for <code>usePaged</code>                      |
| <code>useMemo</code>         | memoise a computed value              | <code>AuthContext</code> context value                                     |
| <code>useContext</code>      | read a Context                        | <code>useAuth()</code> ( <code>AuthContext</code> ), toasts, preferences   |
| <code>useNavigate</code>     | programmatic navigation               | row click → detail page, post-login redirect                               |
| <code>useParams</code>       | read URL params                       | <code>AccountDetailPage</code> ( <code>/portfolio/:id</code> )             |
| <code>useLocation</code>     | current route info                    | <code>Layout</code> breadcrumb, <code>ProtectedRoute</code> redirect state |
| <code>useDispatch</code>     | dispatch Redux actions                | <code>Layout</code> sets unread badge                                      |
| <code>useSelector</code>     | read Redux state                      | topbar bell reads unread count                                             |
| custom <code>usePaged</code> | reusable server-pagination hook       | every list page                                                            |
| custom <code>useAsync</code> | reusable one-shot fetch hook          | detail pages, dashboards                                                   |

\*Custom hooks\* ( `hooks/usePaged.js` ) are how we reuse stateful logic across many pages — a key React strength.

---

## 16. BrowserRouter vs Router vs Routes vs Route

---

- `Router` — the low-level context provider (base for all routers); you rarely use it directly.
- `BrowserRouter` — a `Router` that uses the HTML5 **History API** (clean URLs like `/portfolio/ACC-1` ). Wraps the app once in `main.jsx` .
- `Routes` — the container that looks at the URL and renders the **first matching** `Route` (v6 replaced the old `Switch` ).
- `Route` — one URL-pattern → element mapping ( `<Route path="/legal/:id" element={<LegalCaseDetailPage/>}/>` ); supports nesting (our routes nest under `<Route element={<Layout/>}>` ).
- Also seen: `Navigate` (declarative redirect), `Outlet` (renders nested child routes inside `Layout` ).

```
// main.jsx
<BrowserRouter> ... </BrowserRouter>
// App.jsx
<Routes>
  <Route path="/login" element={<LoginPage/>} />
  <Route element={<ProtectedRoute><Layout/></ProtectedRoute>}>
    <Route path="/portfolio/:id" element={<AccountDetailPage/>} />
  </Route>
</Routes>
```

---

### Quick verbal summary (30 seconds)

---

"DebtPulse is a Spring Boot 3 / Java 21 microservices platform behind a Spring Cloud Gateway, with Eureka discovery, a Config Server, OpenFeign + Resilience4j for inter-service calls, and per-service MySQL via Spring Data JPA. Security is stateless JWT — validated once at the gateway, which injects trusted identity headers that each service turns into a Spring Security context for `@PreAuthorize` . The frontend is a React 18 + Vite SPA using react-router for routing, Context for auth/session, one Redux slice for the notification badge, axios (with token-attach + silent-refresh interceptors) for the API, and react-bootstrap + Recharts for the UI. There's a parallel TypeScript edition kept in parity."

---

\*Generated for the DebtPulse project — interview preparation reference.\*